

ESPectro

Consola portatil de videojuegos basada en ESP32-S3

Memoria Tecnica

Noel Medina & Bernat Figuerola

Processadors Digitals - ESEIAAT, 2026



1. Introduccion

1.1 Motivacion

ESpectro nace con el objetivo de disenar y construir una consola portatil de videojuegos completamente funcional utilizando el microcontrolador ESP32-S3 como procesador principal. El proyecto integra hardware personalizado, firmware en tiempo real con FreeRTOS, conectividad WiFi y un sistema de actualizacion de firmware OTA sin necesidad de cables.

A diferencia de las consolas comerciales, ESpectro esta disenada para ser extensible: cualquier desarrollador puede crear nuevos juegos utilizando la template proporcionada y cargarlos en la consola directamente desde el navegador.

1.2 Objetivos

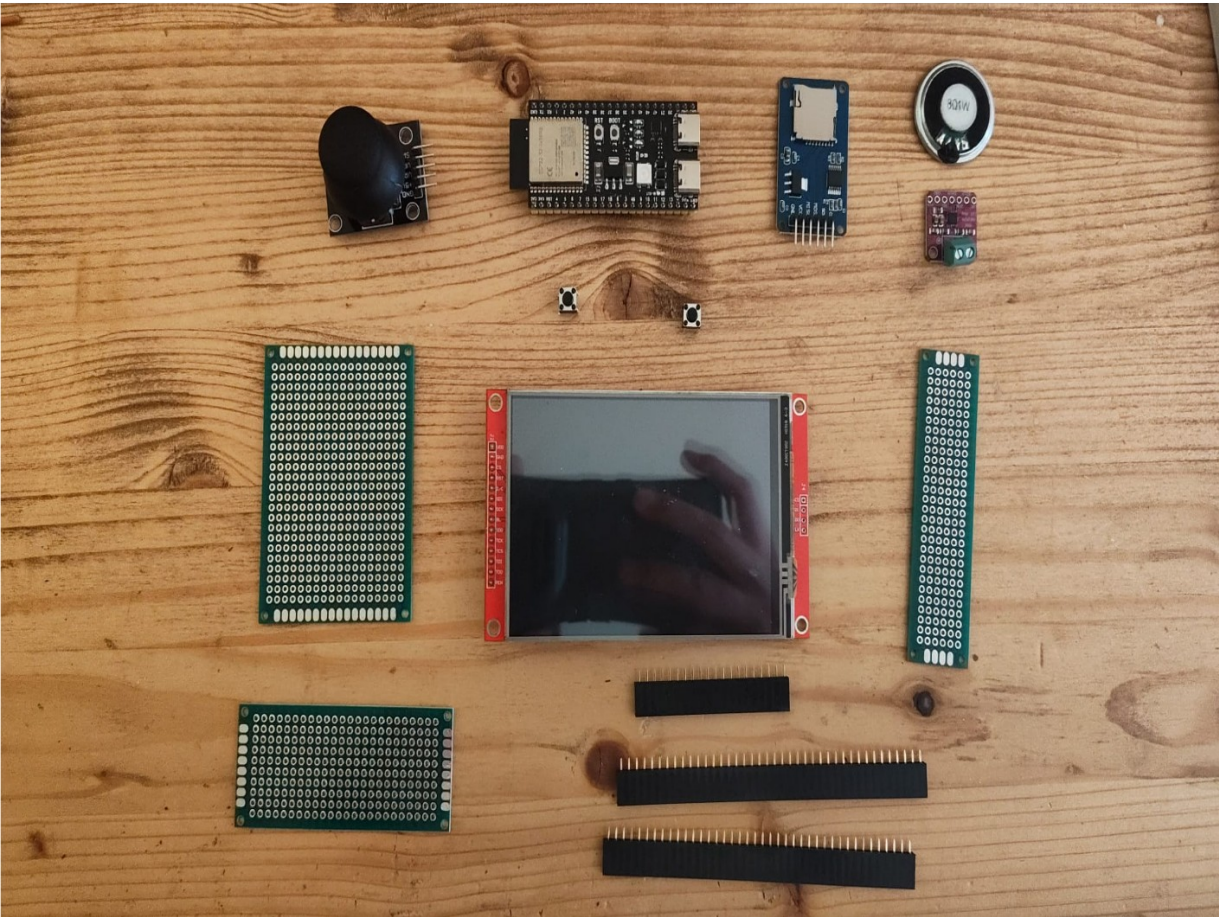
- Disenar e implementar una consola portatil con ESP32-S3 como procesador principal.
- Integrar pantalla TFT, joystick analogico, botones y sistema de audio I2S.
- Implementar FreeRTOS con multiples tareas y mecanismos de sincronizacion.
- Desarrollar un sistema de actualizacion OTA via WiFi con interfaz web.
- Implementar un servidor MCP para integracion con modelos de lenguaje (LLMs).
- Disenar e imprimir en 3D una carcasa para la consola.
- Crear una template reutilizable para el desarrollo de nuevos juegos.

2. Hardware

2.1 Componentes

Componente	Modelo	Interfaz	Funcion
Procesador	ESP32-S3-DevKitC-1 N16R8	—	CPU principal, WiFi, BLE
Pantalla	TFT ILI9488 4"	SPI	Visualizacion del juego
Joystick	HW-504 analogico	ADC	Control de movimiento
Audio	MAX98357A + altavoz 8 ohm	I2S	Efectos de sonido y musica
Botones	2x tactil 6x6mm	GPIO	Boton A y Boton B
Carcasa	PLA impreso en 3D	—	Estructura mecanica

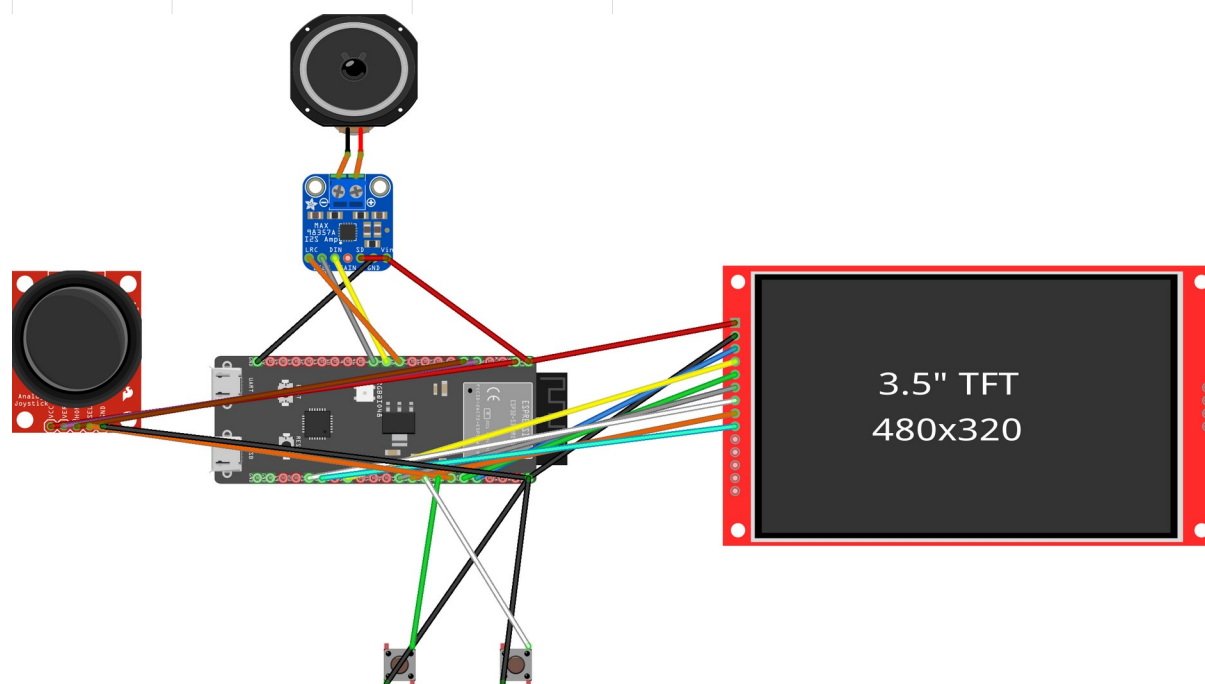
Componentes



Inicialmente ibamos a usar un lector de microSD, pero nos llegaron defectuosos y decidimos no usarlo.

2.2 Mapa de pines

GPIO	Senal	Bus	Componente
38	MOSI	SPI	Pantalla ILI9488
47	SCLK	SPI	Pantalla ILI9488
48	MISO	SPI	Pantalla ILI9488
2	DC	SPI	Pantalla ILI9488
1	CS	SPI	Pantalla ILI9488
0	RST	GPIO	Pantalla ILI9488
39	BL	PWM	Retroiluminacion pantalla
5	VRX	ADC1	Joystick eje X
4	VRY	ADC1	Joystick eje Y
42	SW	GPIO	Boton joystick
40	BTN_A	GPIO	Boton A
41	BTN_B	GPIO	Boton B
8	BCLK	I2S	MAX98357A
16	LRCLK	I2S	MAX98357A
18	DIN	I2S	MAX98357A



fritzing

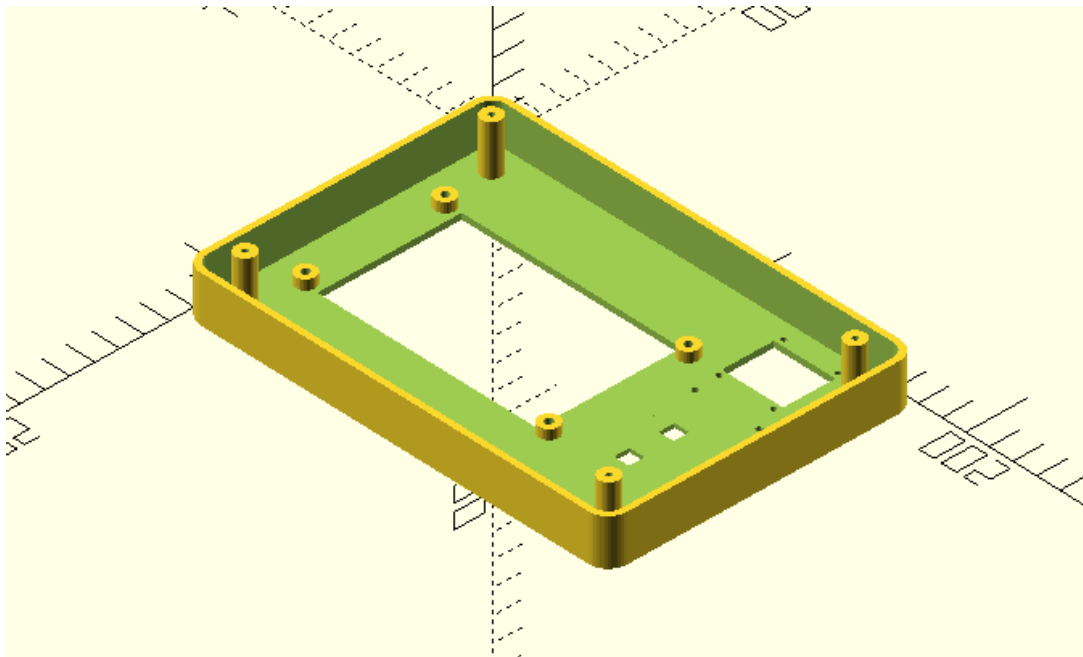
Diagrama hecho con fritzing

2.3 Buses utilizados

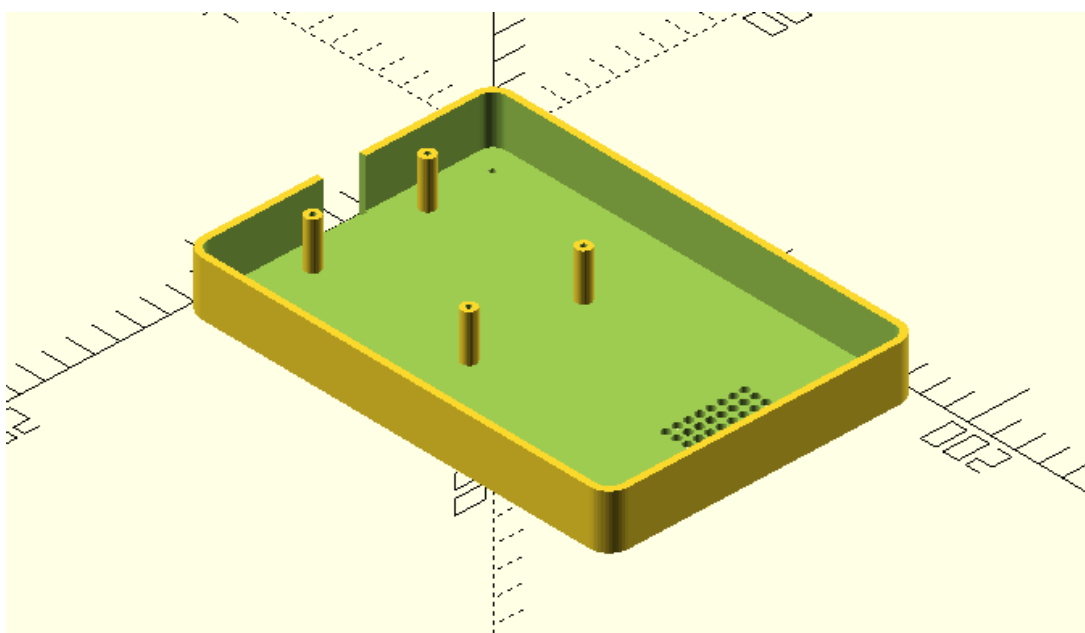
- SPI (SPI2_HOST a 40 MHz): comunicacion con la pantalla ILI9488.
- I2S (I2S_NUM_0 a 44100 Hz): transmision de audio digital al amplificador MAX98357A en formato estereo 16 bits.
- ADC (ADC1, canales GPIO4 y GPIO5): lectura analogica del joystick HW-504. Compatible con WiFi activo en ESP32-S3.

2.4 Carcasa 3D

La carcasa ha sido diseñada en OpenSCAD e impresa en PLA. Consta de dos piezas: frontal (pantalla, joystick y botones) y trasera (ESP32, MAX98357A y altavoz). Dimensiones: 125 x 180 x 24 mm. Las dos piezas se unen con 4 tornillos M2x25.



Parte frontal



Parte trasera

3. Software

3.1 Arquitectura general

El firmware es un binario unico que incluye el launcher y el juego. Al arrancar muestra la splash screen de ESpectro y accede al menu principal, desde donde el usuario puede jugar o acceder al Game Loader para cargar un nuevo juego.

Modulo	Descripcion
Splash screen	Logo ESpectro con jingle musical al arrancar
Menu principal	Muestra el record y las opciones de jugar o cargar
Juego	Logica del juego actual (Road Rush / Tetris)
Game Loader	Servidor web para subir nuevos .bin via WiFi
Dashboard	Estadisticas de todos los juegos en tiempo real
MCP Server	Endpoints para integracion con LLMs via Ollama
NVS	Persistencia de records e historial de partidas

3.2 FreeRTOS - Multitarea

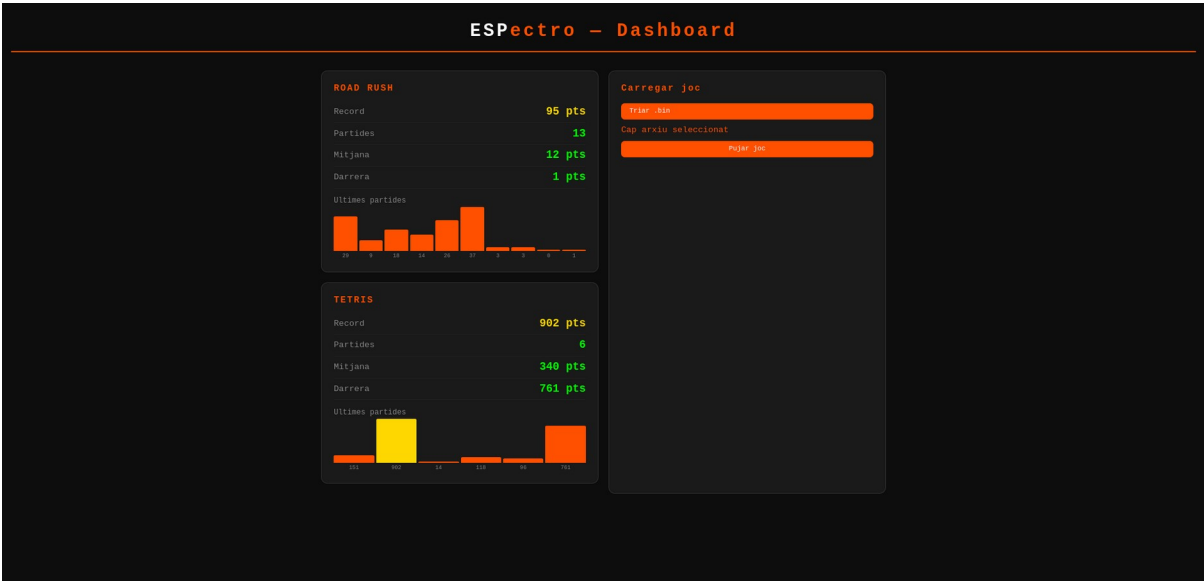
Tarea	Core	Prioridad	Funcion
musicTask	0	2	Reproduce audio I2S de forma continua
wifiTask	0	1	Gestiona el servidor web en segundo plano
loop()	1	—	Logica del juego, menu y controles

Mecanismos de sincronizacion

- audioQueue (Queue): envia eventos de audio desde el juego (core 1) a musicTask (core 0) sin bloquear la ejecucion del juego.
- recordMutex (Mutex): protege el acceso a la NVS entre wifiTask (sirve records al dashboard) y runGame() (escribe records al terminar partida).

3.3 WiFi y dashboard

El WiFi arranca en modo Access Point automaticamente al encender la consola. Red: ESpectro, contrasena: gameloader, IP: 192.168.4.1. El dashboard web muestra todos los juegos registrados con su record maximo, numero de partidas, media de puntuacion y grafica de las ultimas 10 partidas. Se actualiza cada 10 segundos.



Dashboard ESpectro

3.4 OTA - Game Loader

Permite actualizar el firmware sin cables. El usuario pulsa Boton B, se conecta a la red ESpectro, abre <http://192.168.4.1>, selecciona el .bin compilado con PlatformIO y lo sube. El ESP32 lo escribe en flash con Update.h y reinicia automaticamente.

3.5 Persistencia - NVS

Clave NVS	Tipo	Contenido
road_rush	int32	Record maximo del juego Road Rush
road_rush_h	string	JSON con las ultimas 20 puntuaciones
game_list	string	CSV con los juegos registrados

3.6 Servidor MCP

Endpoint	Descripcion
GET /mcp/tools	Lista las tools disponibles
GET /mcp/tools/call?tool=get_records	Records e historial de todos los juegos
GET /mcp/tools/call?tool=get_status	Uptime, memoria libre, IP y version
GET /mcp/tools/call?tool=get_system_info	CPU, flash, PSRAM y version SDK

Se ha desarrollado un script Python (`mcp/mcp_ollama.py`) que actua como puente entre Ollama y los endpoints MCP, inyectando los datos de la consola como contexto al modelo de lenguaje.

3.7 Template para nuevos juegos

Se ha desarrollado una template (PLANTILLA_JUEGO/src/main.cpp) que integra todo el sistema. Para crear un nuevo juego solo hay que rellenar cuatro secciones TODO: RECORD_KEY, titulo del menu, variables globales y la funcion runGame(). Al guardar el primer record, el juego aparece automaticamente en el dashboard.

4. Juegos desarrollados

4.1 Road Rush

Juego de carreras en el que el jugador controla un coche que debe esquivar obstáculos. El joystick controla el movimiento lateral y la velocidad. La puntuación aumenta con cada obstáculo superado. Incluye música ambiente en loop y efectos de sonido vía FreeRTOS.

4.2 Tetris

Implementación del Tetris clásico con las siete piezas estándar. El joystick controla el movimiento lateral y la caída rápida. El Botón A rota la pieza. Puntuación de 100 puntos por línea eliminada con bonus por múltiples líneas simultáneas.

4.3 Blackjack

Implementación del clásico juego de cartas Blackjack con baraja de póker estándar. El jugador compite contra la banca con el objetivo de acercarse a 21 sin pasarse. El Botón A pide carta (Hit) y el Botón B se planta (Stand). Incluye mecánica de apuestas y gestión de bankroll persistente.

4.4 Tres en Ratlla

Implementación del tres en raya contra una inteligencia artificial basada en el algoritmo Minimax, que garantiza el juego óptimo por parte de la máquina. El joystick mueve el cursor por el tablero 3×3 y el Botón A coloca la pieza. La puntuación registra el número de victorias acumuladas.

4.5 Frogger

Recreación del arcade clásico Frogger. El jugador controla una rana que debe cruzar una carretera con coches en movimiento. El joystick controla el movimiento en las cuatro direcciones. La puntuación aumenta con cada cruce completado y la dificultad se incrementa progresivamente.

4.6 Snake

Implementación del Snake clásico. El jugador controla una serpiente que crece al comer manzanas y debe evitar chocar contra las paredes o contra su propio cuerpo. El joystick controla la dirección. La puntuación aumenta con cada manzana comida.

4.7 Flappy Bird

Recreación del popular juego Flappy Bird. El jugador controla un pájaro que debe esquivar tuberías pulsando el Botón A para aletear. La física de gravedad hace que el pájaro caiga continuamente. La puntuación aumenta con cada tubería superada.

4.8 Buscaminas

Implementación del Buscaminas clásico con tablero configurable. El joystick mueve el cursor por el tablero y el Botón A revela una casilla. El Botón B marca o desmarca una mina sospechosa con bandera. La puntuación se basa en el tiempo empleado en completar el tablero.

5. Resultados

5.1 Requisitos cumplidos

Requisito	Estado	Implementacion
ESP32 como procesador	SI	ESP32-S3-DevKitC-1 N16R8
Conectividad WiFi	SI	AP siempre activo (ESpectro)
OTA operativa	SI	HTTP POST con Update.h
Display	SI	TFT ILI9488 4" via SPI
Sensores (ADC/SPI)	SI	Joystick ADC1, pantalla SPI
FreeRTOS 2+ tareas	SI	3 tareas: music, wifi, loop
Sincronizacion RTOS	SI	Queue + Mutex
2+ buses distintos	SI	SPI + I2S + ADC
PlatformIO reproducible	SI	platformio.ini documentado
MCP (innovacion)	SI	4 endpoints + puente Ollama

5.2 Rendimiento

- Framerate del juego: ~50 fps (frame cada 20 ms en core 1).
- Latencia audio: < 10 ms gracias a la arquitectura de colas FreeRTOS.
- Tiempo de arranque: ~3 segundos hasta el menu principal.
- Tiempo de carga OTA: ~30 segundos para un firmware de 1.2 MB.
- Memoria heap libre en juego: ~120 KB de los 320 KB disponibles.

6. Conclusiones

6.1 Logros

El proyecto ESpectro ha cumplido todos los requisitos técnicos de la asignatura e incorpora elementos innovadores como el servidor MCP y la integración con modelos de lenguaje. La arquitectura modular basada en la template permite extender el sistema con nuevos juegos de forma sencilla, lo que ha quedado demostrado con el desarrollo de dos juegos completamente funcionales.

6.2 Dificultades encontradas

- Compatibilidad entre LovyanGFX y algunos headers de ESP-IDF: resuelto usando `nvs_flash.h` directamente.
- Módulo de microSD defectuoso / tarjetas microSD defectuosas; descartamos el uso del módulo.
- Inestabilidad de la pantalla en el arranque sin reset hardware: resuelto conectando RST al GPIO0.
- Saturación del audio con `playTone()` sincrónico: resuelto mediante cola FreeRTOS (`audioQueue`).
- Limitaciones de VRAM de la GPU en el PC para Ollama: resuelto forzando ejecución en CPU.

6.3 Trabajo futuro

- Añadir autenticación al Game Loader para mayor seguridad.
- Soporte para múltiples usuarios en el dashboard.
- Ampliar el servidor MCP con una tool de control remoto del juego.
- Publicar la template como librería PlatformIO.

7. Referencias

- Espressif Systems. ESP32-S3 Technical Reference Manual. <https://www.espressif.com>
- LovyanGFX. Documentacion oficial. <https://github.com/lovyan03/LovyanGFX>
- Anthropic. Model Context Protocol (MCP). <https://modelcontextprotocol.io>
- FreeRTOS. Reference Manual. <https://www.freertos.org>
- PlatformIO. Documentacion ESP32. <https://docs.platformio.org>
- Ollama. Documentacion oficial. <https://ollama.com>